

Some principal ideas:

From startup most important information's are presented on the USB serial port. You can activate the `#define TEST` mode on top of the main .ino file and will get much more detailed infos.

I have quite a lot of devices, use a separated IOT network and insist on defined addresses for all devices. Therefore, I do include the address in the mqtt devices name. Having my own Node Red implementation, I defined a mqtt naming convention like "**device-type-IP**". ALL sensor data are in mqtt **/sen/.....** by using a similar structure like the Shelly-device convention: F.x. the first **temperature** sensor arrives as "**device-type-IP**/**sen/temp/0**. **esp/C3-SEN-033/sen/temp/0**
The type defined **number** is automatically calculated according to its appearance in the startup sequence starting at 0. When you request via the "**device-name**"/command **?** you see under **/sensors** the **number** in the **<type x>** .

MQTT definitions:

```
#define MyIP "033" // Device ID

//Wifi definitions of this device
IPAddress staticIP(192,168,167,atoi(MyIP)); // This device has IP
IPAddress subnet(255,255,255,0); // Network subnet size
IPAddress gateway(192,168,167,1); // IOT Network router IP

// Raspberri Pi Mosquitto MQTT Broker definitions
#define mqtt_server "192.168.167.20" // IOT MQTT server IP
#define mqtt_user "admin"
#define mqtt_password "admin"
#define mqtt_port 1883
#define WiFi_timeout 101 // How many times to try before give up
#define mqtt_timeout 11 // How many times to try before try Wifi reconnect

// MQTT Topics
#define mytype "esp/C3-SEN-" // Client Typ
#define iamclient mytype MyIP // Client name
#define in_topic iamclient "/command" // This common input is received from MQTT
#define out_param iamclient "/signal" // Wifi signal strength is send to MQTT
#define out_status iamclient "/status" // This is a general message send to MQTT
#define out_watchdog iamclient "/watchdog" // A watchdog bit send to MQTT
#define out_sensors iamclient "/sensors" // This is a list of usable external sensors send to MQTT
#define out_topic iamclient "/loop" // This helper debug variable can be send to MQTT
#define mqtt_debug iamclient "/debug" // This is send to MQTT Debug only
#define mqtt_out_sen iamclient "/sen" // This is a generic starter descriptor to send to MQTT
#define mqtt_out_qual iamclient "/sen/qual" // Measurement quality is send to MQTT
```

MQTT command decoding is made via (around line 500)

```
void callback(char* topic, uint8_t* payload, unsigned int length) {
```

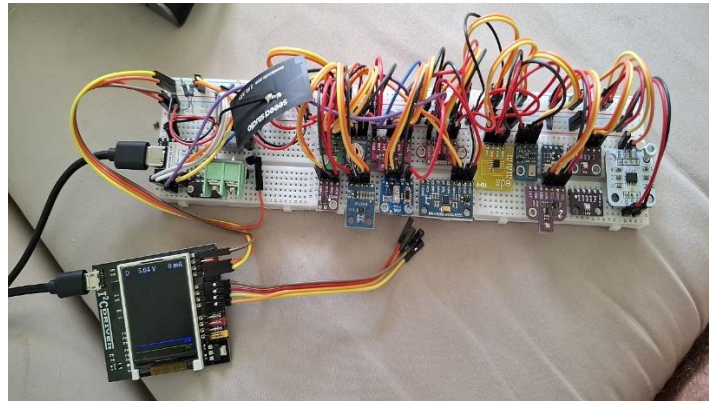
Command **?** requests the device status.

Command **U xx** is low used in this device, it changes mqtt timings.

Command **X** restarts the device.

Sensor definition

The device is intentional only configured via the `credentials.h` file.



Especially what sensors you want to implement is defined there. Every sensor has his: a.) Definition / b.) Startup sequence / and c.) Read cycle coded in his .h file.

Device definitions:

```
// Allowed are: DHT 11 or DHT 12 or DHT 22 (AM2302), AM2321 or DHT 21 (AM2301)
#define DHTtyp DHT22 // See above

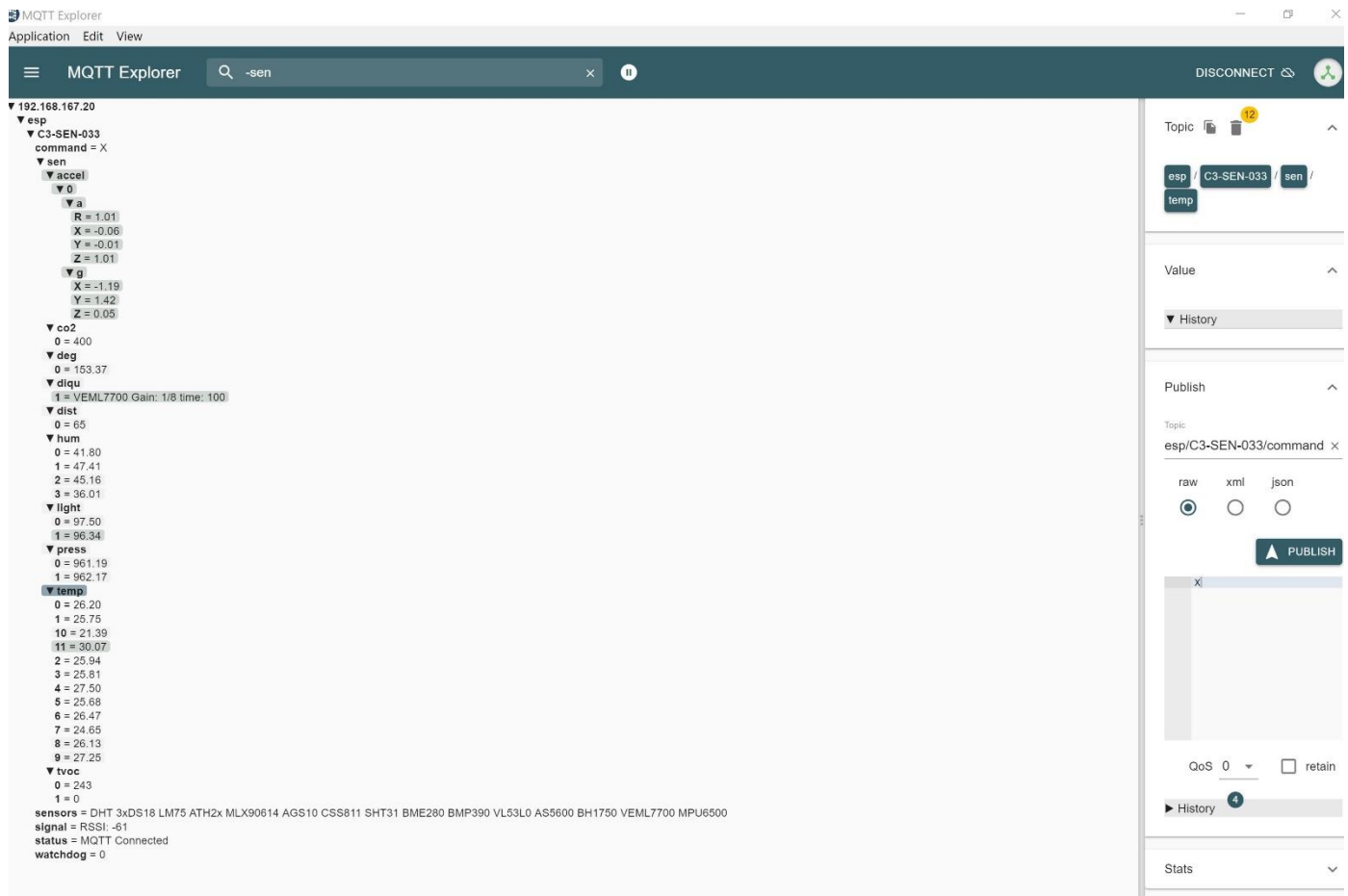
#define enableDHT // Enable DHT T&H sensor
#define enableDS // Enable multiple DS18 T sensors
#define enableLM75 // I2C_ADDRESS 0x48 Enable LM75 T sensor
#define enableAHT // I2C_ADDRESS 0x38 Enable AHT21 T&H sensor
#define enableSHT31 // I2C_ADDRESS 0x44 Enable SHT31 T&H sensor
#define enableMLX // I2C_ADDRESS 0x5A Enable MLX90614 nontouch T
#define enableAGS10 // I2C_ADDRESS 0x1A Enable AGS10 tvoc sensor
#define enableCCS // I2C_ADDRESS (nc)-0x5A (VCC)-0x5B Enable CCS
//#define enableENS // I2C_ADDRESS 0x53 Enable ENS160 tvoc senso
//#define enableSGP30 // I2C_ADDRESS 0x58 Enable SGP30 tvoc sensor
//#define enableBMP18 // EITHER BMP/E-3xx // I2C_ADDRESS 0x77 Enable BMP180 T & Baropr
#define enableBMP39 // OR BMP 180 // I2C_ADDRESS 0x77 Enable BMP/E-3XX T&H & Baropr
//#define enableBMP28 // EITHER BMP 280 // I2C_ADDRESS 0x76 Enable BMP280 T & Baropr
#define enableBME28 // OR BME 280 // I2C_ADDRESS 0x76 Enable BME280 T&H & Baropr
#define enableVL53L0 // I2C_ADDRESS 0x29 Enable VL53L0X tof1 sensor
//#define enableVL6180 // EITHER AS5600 // I2C_ADDRESS 0x36 Enable VL6180X tof1 sens
#define enableAS56 // OR VL6180 // I2C_ADDRESS 0x36 Enable AS5600 angle sensor
#define enableBH17 // I2C_ADDRESS (nc)-0x23 (VCC)-0x5C Light sens
#define enableVEML // I2C_ADDRESS 0x10 Light sensor VEML7700
#define enableMPU65 // I2C_ADDRESS (nc)-0x68 (VCC)-0x69 6 axis sens
```

MQTT examples:

These images are taken from my test configuration and proof that a single esp can run a senseless great number of sensors on the same 4.7k loaded bus in the same time. It also proofs that the auto-numbering functions very well.

At startup the mqtt variable `/sensors` shows the running sensors (if a configured sensor is not working correct it is shown in the mqtt variable `/status`. If a sensor cannot be started successful after reset it will be excluded from all activities) After a `/command ?` request the `/sensors` variable shows for each working sensor its type and number: `<type x >` .

After restart:



The screenshot shows the MQTT Explorer interface after a restart. The left pane displays a tree view of the MQTT topics and their values. The right pane shows the 'Publish' section with the topic 'esp/C3-SEN-033/command' and the 'raw' format selected. The 'History' section shows a list of messages.

MQTT Explorer

Application Edit View

MQTT Explorer -sen

DISCONNECT

▼ 192.168.167.20

- ▼ esp
 - ▼ C3-SEN-033
 - command = X
 - ▼ sen
 - ▼ accel
 - ▼ 0
 - ▼ a
 - R = 1.01
 - X = -0.06
 - Y = -0.01
 - Z = 1.01
 - ▼ g
 - X = -1.19
 - Y = 1.42
 - Z = 0.05
 - ▼ co2
 - 0 = 400
 - ▼ deg
 - 0 = 153.37
 - ▼ diqu
 - 1 = VEML7700 Gain: 1/8 time: 100
 - ▼ dist
 - 0 = 65
 - ▼ hum
 - 0 = 41.80
 - 1 = 47.41
 - 2 = 45.16
 - 3 = 36.01
 - ▼ light
 - 0 = 97.50
 - 1 = 96.34
 - ▼ press
 - 0 = 961.19
 - 1 = 962.17
 - ▼ temp
 - 0 = 26.20
 - 1 = 25.75
 - 10 = 21.39
 - 11 = 30.07
 - 2 = 25.94
 - 3 = 25.81
 - 4 = 27.50
 - 5 = 25.68
 - 6 = 26.47
 - 7 = 24.65
 - 8 = 26.13
 - 9 = 27.25
 - ▼ tvoc
 - 0 = 243
 - 1 = 0

sensors = DHT 3xDS18 LM75 ATH2x MLX90614 AGS10 CSS811 SHT31 BME280 BMP390 VL53L0 AS5600 BH1750 VEML7700 MPU6500
signal = RSSI: -61
status = MQTT Connected
watchdog = 0

Topic: esp / C3-SEN-033 / sen / temp

Value

▼ History

Publish

Topic: esp/C3-SEN-033/command

raw xml json

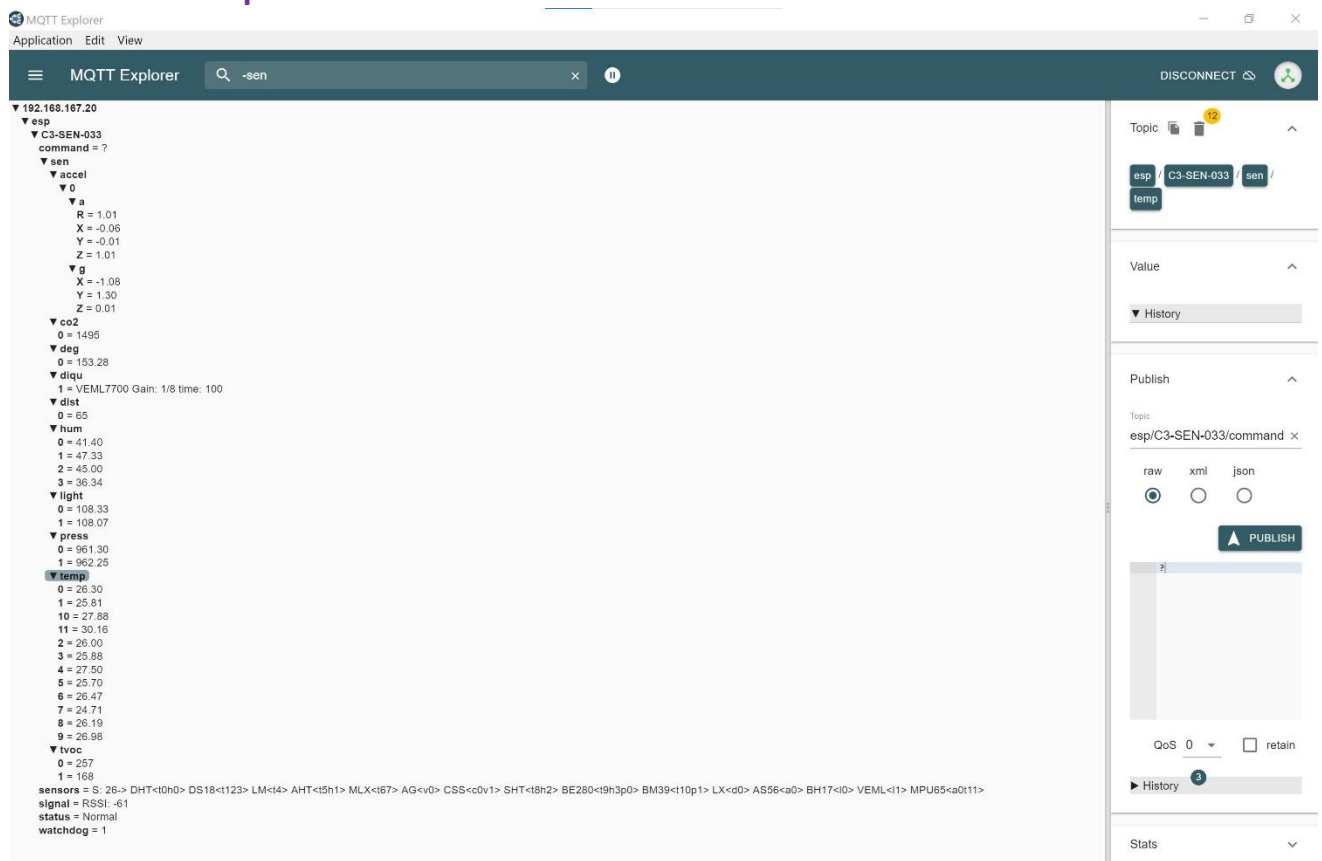
PUBLISH

QoS 0 retain

History 4

Stats

After status request:



The screenshot shows the MQTT Explorer interface after a status request. The left pane displays a tree view of the MQTT topics and their values. The right pane shows the 'Publish' section with the topic 'esp/C3-SEN-033/command' and the 'raw' format selected. The 'History' section shows a list of messages.

MQTT Explorer

Application Edit View

MQTT Explorer -sen

DISCONNECT

▼ 192.168.167.20

- ▼ esp
 - ▼ C3-SEN-033
 - command = ?
 - ▼ sen
 - ▼ accel
 - ▼ 0
 - ▼ a
 - R = 1.01
 - X = -0.06
 - Y = -0.01
 - Z = 1.01
 - ▼ g
 - X = -1.08
 - Y = 1.30
 - Z = 0.01
 - ▼ co2
 - 0 = 1495
 - ▼ deg
 - 0 = 153.28
 - ▼ diqu
 - 1 = VEML7700 Gain: 1/8 time: 100
 - ▼ dist
 - 0 = 65
 - ▼ hum
 - 0 = 41.40
 - 1 = 47.33
 - 2 = 45.00
 - 3 = 36.34
 - ▼ light
 - 0 = 108.33
 - 1 = 108.07
 - ▼ press
 - 0 = 961.30
 - 1 = 962.25
 - ▼ temp
 - 0 = 26.30
 - 1 = 25.81
 - 10 = 27.88
 - 11 = 30.16
 - 2 = 26.00
 - 3 = 25.88
 - 4 = 27.50
 - 5 = 25.70
 - 6 = 26.47
 - 7 = 24.71
 - 8 = 26.19
 - 9 = 26.98
 - ▼ tvoc
 - 0 = 257
 - 1 = 168

sensors = S: 26-> DHT<10h0> DS18<1123> LM<14> AHT<15h1> MLX<167> AG<v0> CSS<c0v1> SHT<18h2> BE280<19h3p0> BM39<110p1> LX<d0> AS56<a0> BH17<10> VEML<1> MPU85<a0t11>
signal = RSSI: -61
status = Normal
watchdog = 1

Topic: esp / C3-SEN-033 / sen / temp

Value

▼ History

Publish

Topic: esp/C3-SEN-033/command

raw xml json

PUBLISH

QoS 0 retain

History 3

Stats